

1. Insert the following elements 150, 250, 350, 450, 550 into the double linked list and delete the elements 150 and 250 from the double linked list. Show the linked structure using a node diagram before and after delete operations.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
int data;
```

```
struct node *prev, *next;
```

```
};
```

```
struct node *head = NULL *newnode,
```

```
*temp;
```

```
void insert(int x)
```

```
{
```

```
newnode = (struct
```

```
node*) malloc (sizeof (struct node));
```

```
newnode->data = x;
```

```
newnode->next = NULL;
```

```
if (head == NULL)
```

```
{
```

```
newnode → prev = NULL;
```

```
head = newnode;
```

```
}
```

```
else
```

```
{
```

```
temp = head;
```

```
while (temp → next != NULL)
```

```
temp = temp → next;
```

```
temp → next = newnode;
```

```
newnode → prev = temp;
```

```
}
```

```
}
```

```
void delete (int x)
```

```
{
```

```
temp = head;
```

```
while (temp != NULL && temp → data != x)
```

```
temp = temp → next;
```

```
if (temp == NULL)
```

```
return;
```

```
if (temp == head)
```

```
{
```

```
head = temp → next;
```

```
if (head != NULL)
```

```
head → prev = NULL;
```

```

}
else
{
    temp → prev → next = temp → next;
    if (temp → next != NULL)
        temp → next → prev = temp → prev;
}
free(temp);
}

```

```

void display()

```

```

{
    temp = head;
    while (temp != NULL)
    {
        printf("%d", temp → data);

        temp = temp → next;
    }
    printf("\n");
}

```

```

int main()

```

```

{
    insert(150);
    insert(250);
}

```

```
insert (350);
```

```
insert (450);
```

```
insert (550);
```

```
printf("Before Deletion:");
```

```
display();
```

```
delete (150);
```

```
delete (250);
```

```
printf ("After Deletion:");
```

```
display();
```

```
return 0;
```

```
}
```

Output:-

Elements inserted : 150, 250, 350, 450, 550

Before Deletion : 150 250 350 450 550

After Deletion : 350 450 550

NODE DIAGRAM:-

Before Deletion:-

$\text{NULL} \leftarrow [150] \rightleftarrows [250] \rightleftarrows [350] \rightleftarrows [450] \rightleftarrows [550] \rightarrow \text{NULL}$

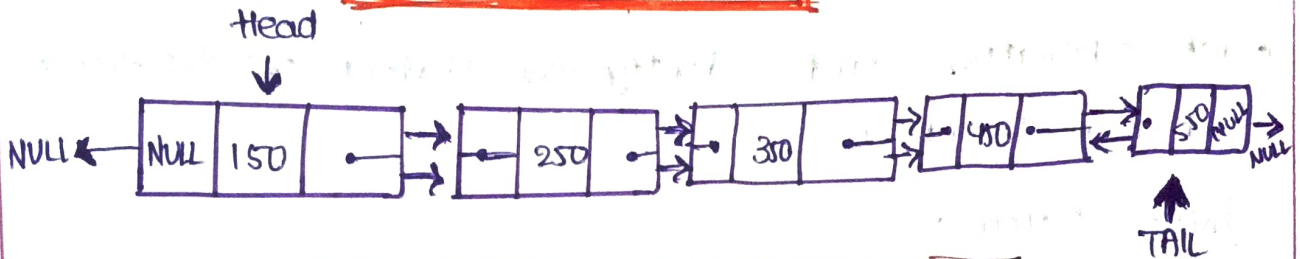
After Deleting 150:-

$\text{NULL} \leftarrow [250] \rightleftarrows [350] \rightleftarrows [450] \rightleftarrows [550] \rightarrow \text{NULL}$

After Deleting 250 (Final list)

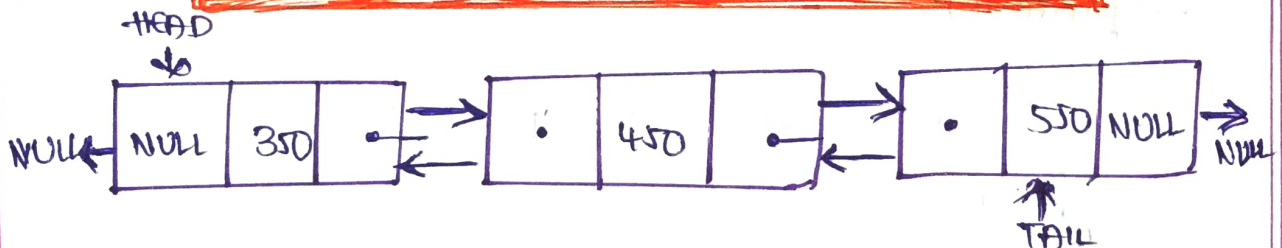
NULL $\leftarrow$ [350] $\rightleftarrows$ [450] $\rightleftarrows$ [550] $\rightarrow$ NULL

BEFORE DELETION



• = pointer $\rightarrow$ = Next $\leftarrow$ = previous

After deleting 150 and 350



• = pointer $\rightarrow$ = Next $\leftarrow$ = previous

Notes

- $\Rightarrow$ Each node has three parts : previous / data / Next
- $\Rightarrow$ Arrows in red show the links (connection).

2. A circular queue has a size of 6. Execute the following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70). Draw the queue after each operation and identify the values of the front and rear pointers.

Initial Queue:-

0	1	2	3	4	5
---	---	---	---	---	---

Front = -1, Rear = -1

1) Enqueue(10):-

0	1	2	3	4	5
10					

Front = 0, Rear = 0

2) Enqueue(20):-

0	1	2	3	4	5
10	20				

Front = 0, Rear = 1

3) Enqueue (30):-

0	1	2	3	4	5
10	20	30			

Front = 0 , Rear = 2

4) Dequeue ():- (Removes 10)

0	1	2	3	4	5
	20	30			

Front = 1 , Rear = 2

5) Enqueue (40):-

0	1	2	3	4	5
	20	30	40		

Front = 1 , Rear = 3

6) Enqueue (50):-

0	1	2	3	4	5
	20	30	40	50	

Front = 1 , Rear = 4

7) Enqueue (60):-

0	1	2	3	4	5
	20	30	40	50	60

Front = 1, Rear = 5

8) Enqueue (70):- (Rear wraps around to index 0)

0	1	2	3	4	5
70	20	30	40	50	60

Front = 1, Rear = 0

Final Queue:-

		Front ↓			
70	20	30	40	50	60
↑ Rear					